

DBMS Lab Assignment - 4

Title: Query formation using nested update, insert, and delete statements with the GROUP BY and HAVING clauses, and SQL Views.

Description:

Aggregate functions are performed on the columns values. But if we want to divide values stored in a single column in groups and then perform certain actions, then we have to use Group By clause.

GROUP BY is use to **group** values from a column, and, if you wish, performs calculations on that column. You can use COUNT, SUM, AVG, etc., functions on the grouped column.

Syntax

The syntax for the GROUP BY clause in MySQL is:

```
SELECT expression1, expression2, ... expression_n,  
       aggregate_function (expression)  
FROM tables  
[WHERE conditions]  
GROUP BY expression1, expression2, ... expression_n;
```

Parameters or Arguments

expression1, expression2, ... expression_n

The expressions that are not encapsulated within an aggregate function and must be included in the GROUP BY clause.

aggregate_function

A function such as SUM, COUNT, MIN, MAX, or AVG functions.

tables

The tables that you wish to retrieve records from. There must be at least one table listed in the FROM clause.

WHERE conditions

Optional. The conditions that must be met for the records to be selected.

This GROUP BY example uses the MAX function to return the name of each department and the maximum salary in the department.

```
SELECT department, MAX(salary) AS "Highest salary"  
FROM employees  
GROUP BY department;
```

Having Clause: - If we want to apply certain condition on which values should get included in the group, we have to use the "Having Clause".

Syntax

The syntax for the HAVING Clause in MySQL is:

```
SELECT expression1, expression2, ... expression_n,  
       aggregate_function (expression)  
FROM tables  
[WHERE conditions]  
GROUP BY expression1, expression2, ... expression_n  
HAVING condition;
```

Parameters or Arguments

aggregate_function

A function such as SUM, COUNT, MIN, MAX, or AVG functions.

expression1, expression2, ... expression_n

The expressions that are not encapsulated within an aggregate function and must be included in the GROUP BY clause.

WHERE conditions

Optional. These are the conditions for the records to be selected.

HAVING condition

This is a further condition applied only to the aggregated results to restrict the groups of returned rows. Only those groups whose condition evaluates to TRUE will be included in the result set.

Example - Using SUM function

Let's look at a MySQL HAVING clause example that uses the [SUM function](#).

You could also use the [SUM function](#) to return the name of the product and the total quantity (for that product). The MySQL HAVING clause will filter the results so that only products with a total quantity greater than 10 will be returned.

```
SELECT product, SUM(quantity) AS "Total quantity"  
FROM order_details  
GROUP BY product  
HAVING SUM(quantity) > 10;
```

Order By:-

The MySQL ORDER BY clause is used to sort the records in your result set.

Syntax

The syntax for the ORDER BY clause in MySQL is:

```
SELECT expressions
FROM tables
[WHERE conditions]
ORDER BY expression [ ASC | DESC ];
```

Example:

```
SELECT last_name, first_name, city
FROM contacts
WHERE last_name = 'Johnson'
ORDER BY city DESC;
```

This MySQL ORDER BY example would return all records sorted by the *city* field in descending order.

We can also set a limit on the number of values which we want to be returned by the execution of order by query.

For example:

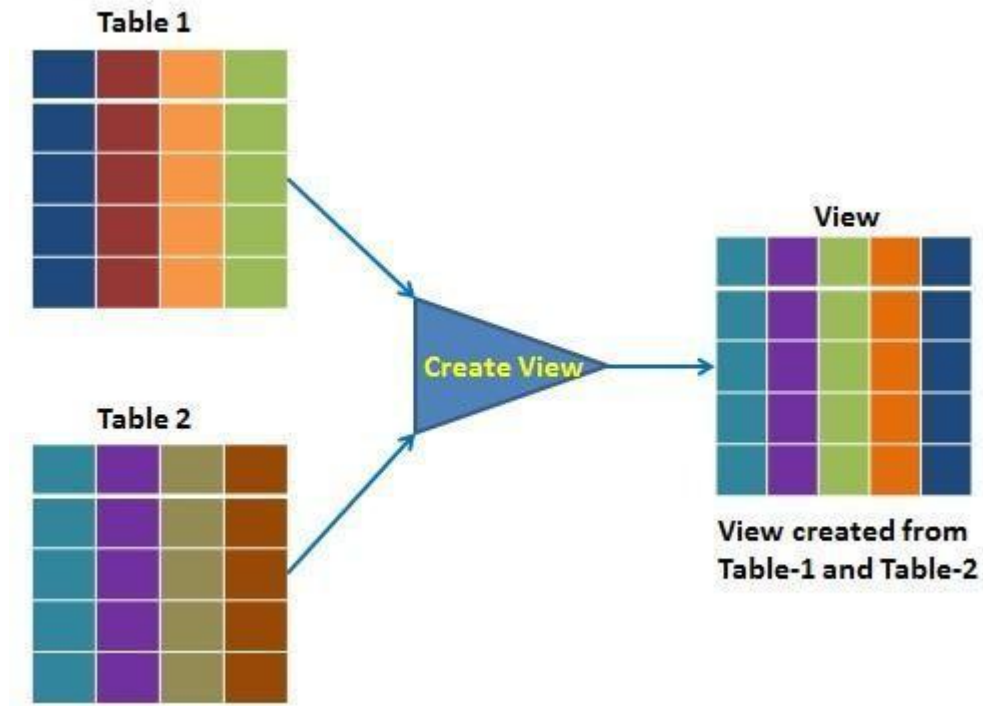
```
SELECT contact_id, last_name, first_name
FROM contacts
WHERE website = 'TechOnTheNet.com'
ORDER BY contact_id DESC
LIMIT 5;
```

This MySQL SELECT LIMIT example would select the first 5 records from the *contacts* table where the *website* is 'TechOnTheNet.com'. Note that the results are sorted by *contact_id* in descending order so this means that the 5 largest *contact_id* values will be returned by the SELECT LIMIT statement.

SQL Views

In SQL, a view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.



You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

CREATE VIEW Syntax

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Note: A view always shows up-to-date data! The database engine recreates the data, using the view's SQL statement, every time a user queries a view.

Types of Views

- **Simple View:** A view based on only a single table, which doesn't contain GROUP BY clause and any functions.
- **Complex View:** A view based on multiple tables, which contain GROUP BY clause and functions.
- **Inline View:** A view based on a subquery in FROM Clause, that subquery creates a temporary table and simplifies the complex query.
- **Materialized View:** A view that stores the definition as well as data. It creates replicas of data by storing it physically.

Assignment 4 Lab Questions

Perform following queries using Group by, having and Order By

1. Give library wise book details.

```
mysql> SELECT s.Stuid, s.Sname, s.email
-> FROM student s
-> JOIN department d ON s.deptid = d.Deptid
-> WHERE d.deptname = 'IT';
```

Stuid	Sname	email
805	Rohan Deshmukh	rohan.d@student.siu.edu

1 row in set (0.01 sec)

2. Give bookwise total copies which are available.

```
mysql> SELECT Lid, COUNT(Bid) AS total_books
-> FROM BOOKS
-> GROUP BY Lid
-> ORDER BY Lid ASC;
```

Lid	total_books
101	3
102	2
103	1
104	1
106	1
107	1
108	1

7 rows in set (0.00 sec)

3. Which library has total copies more than 100?

```
mysql> SELECT blid AS Lid, COUNT(bnid) AS total_copies
-> FROM Noofcopies
-> GROUP BY blid
-> HAVING COUNT(bnid) > 100
-> ORDER BY total_copies DESC;
Empty set (0.00 sec)
```

4. Give institute wise department details.

```
mysql> SELECT Iname AS institute_name, COUNT(Deptid) AS total_departments
-> FROM DEPARTMENT
-> GROUP BY Iname
-> ORDER BY Iname ASC;
```

institute_name	total_departments
SCHC	1
SIBM	1
SICSR	1
SIT	4
SLS	1
SSBS	1
SSLA	1

7 rows in set (0.00 sec)

5. Give citywise seller details.

```
mysql> SELECT city, COUNT(Sid) AS total_sellers
-> FROM SELLER
-> GROUP BY city
-> ORDER BY city ASC;
```

city	total_sellers
Bangalore	1
Delhi	1
Hyderabad	1
Mumbai	1
Nagpur	1
Nashik	1
Noida	1
Pune	3

8 rows in set (0.00 sec)

6. Give author wise book details that have authored more than 2 books.

```
mysql> SELECT Aid, COUNT(Bid) AS books_authored
-> FROM Writes
-> GROUP BY Aid
-> HAVING COUNT(Bid) > 1
-> ORDER BY books_authored DESC;
```

Aid	books_authored
302	2

1 row in set (0.00 sec)

7. Give book details library wise whose price is less than 1000

```
mysql> SELECT Lid, COUNT(Bid) AS total_books
-> FROM BOOKS
-> WHERE Price < 1000
-> GROUP BY Lid
-> ORDER BY Lid ASC;
+-----+-----+
| Lid | total_books |
+-----+-----+
| 101 |          2 |
| 103 |          1 |
| 106 |          1 |
+-----+-----+
3 rows in set (0.01 sec)
```

8. Give department wise staff details.

```
mysql> SELECT deptid, COUNT(Stid) AS total_staff
-> FROM STAFF
-> GROUP BY deptid
-> ORDER BY deptid ASC;
+-----+-----+
| deptid | total_staff |
+-----+-----+
|    601 |          1 |
|    602 |          1 |
|    603 |          1 |
|    604 |          1 |
|    605 |          1 |
|    606 |          1 |
|    607 |          1 |
|    608 |          1 |
|    609 |          1 |
|    610 |          1 |
+-----+-----+
10 rows in set (0.00 sec)
```

9. How many books are issued library wise

```
mysql> SELECT lid, COUNT(Issueid) AS total_issued_books
-> FROM ISSUE
-> GROUP BY lid
-> ORDER BY total_issued_books DESC;
+-----+-----+
| lid | total_issued_books |
+-----+-----+
| 101 |          2 |
| 102 |          1 |
| 103 |          1 |
| 104 |          1 |
| 106 |          1 |
| 107 |          1 |
| 108 |          1 |
| 109 |          1 |
| 110 |          1 |
+-----+-----+
9 rows in set (0.00 sec)
```

10. Give purchase details publisher wise.

```
mysql> SELECT pid, COUNT(pid) AS total_purchases, SUM(totalcost) AS total_expenditure
-> FROM PURCHASE
-> GROUP BY pid
-> ORDER BY total_expenditure DESC;
```

pid	total_purchases	total_expenditure
406	1	67500.00
401	3	52600.00
403	1	50000.00
407	1	39000.00
408	1	16200.00
404	1	11985.00
405	1	5950.00
402	1	3750.00

8 rows in set (0.01 sec)

11. Display books in a descending order of their cost.

```
mysql> SELECT Bid, Bname, Price
-> FROM BOOKS
-> ORDER BY Price DESC;
```

Bid	Bname	Price
210	Constitutional Law	13500.00
204	Advanced Artificial Intelligence	12500.00
206	Introduction to Algorithms	11000.00
208	Cloud Computing Essentials	6500.00
209	Business Analytics	1800.00
203	Data Structures Using C	1200.00
201	Database Management Systems	950.00
205	Operating System Concepts	850.00
207	Software Engineering	799.00
202	Computer Networks	750.00

10 rows in set (0.00 sec)

Perform a study assignment on SQL Views which should include following details:

12. Need of SQL Views

- A **View** in SQL is a virtual table derived from a result-set of a database query. It contains rows and columns just like a real table, but it does not store physical data on the disk (except in materialized views).
- **Data Security & Privacy:** Views allow administrators to restrict data access by exposing only specific columns or rows to unauthorized users without revealing the underlying base tables.
- **Simplification of Complex Queries:** Complex logic, including multi-table JOINS, subqueries, and aggregations, can be saved inside a view so users can query it using simple SELECT statements.
- **Data Independence (Abstraction):** Views provide a logical layer of abstraction. If the schema of underlying tables changes (e.g., columns are renamed or split),

updating the view definition keeps external applications from breaking.

13. How to implement and use views in MYSQL.

Create view :-

```
CREATE VIEW view_name AS  
SELECT column1, column2, column3  
FROM table_name  
WHERE condition;
```

Quering view :-

```
SELECT * FROM view_name  
WHERE column1 = 'value';
```

Modifying view :-

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column1, column2  
FROM table_name  
WHERE new_condition;
```

14. Write a view to select all books of SIT library whose cost is less than 1000/-

```
CREATE VIEW sit_books_under_1000 AS  
SELECT  
    b.Bid AS book_id,  
    b.Bname AS book_title,  
    b.Price AS price,  
    l.Lname AS library_name  
FROM BOOKS b  
JOIN library l ON b.Lid = l.Lid  
WHERE l.Lname = 'SIT'  
AND b.Price < 1000;
```

Name :- Parth Bhayani

Prn No :- 26070122512

Batch :- B3

Output :-

```
SELECT * FROM sit_books_under_1000;
```

Conclusion: Thus we have implemented Query formation.